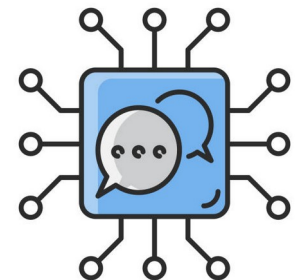


Word2Vec Model

Tushar B. Kute,
<http://tusharkute.com>



How Do Computers Read?

- The Challenge: Computers are mathematical machines. They understand numbers, not text.
 - The Question: How do we convert a word like "Apple" into a set of numbers that a computer can process?
 - The Requirement: We don't just want unique IDs (like serial numbers). We want the numbers to capture the meaning of the word.

The "Switchboard" Method (One-Hot Encoding)

- Concept: Imagine a vocabulary of 5 words: [King, Queen, Man, Woman, Apple].
- Representation: We create a list of zeros and turn on only one switch for the specific word.
 - King: [1, 0, 0, 0, 0]
 - Queen: [0, 1, 0, 0, 0]
 - Apple: [0, 0, 0, 0, 1]
- The Problem:
 - No Meaning: "King" is mathematically just as different from "Queen" as it is from "Apple". The relationship is lost.
 - Space Waste: If English has 100,000 words, every vector is 99.999% zeros.

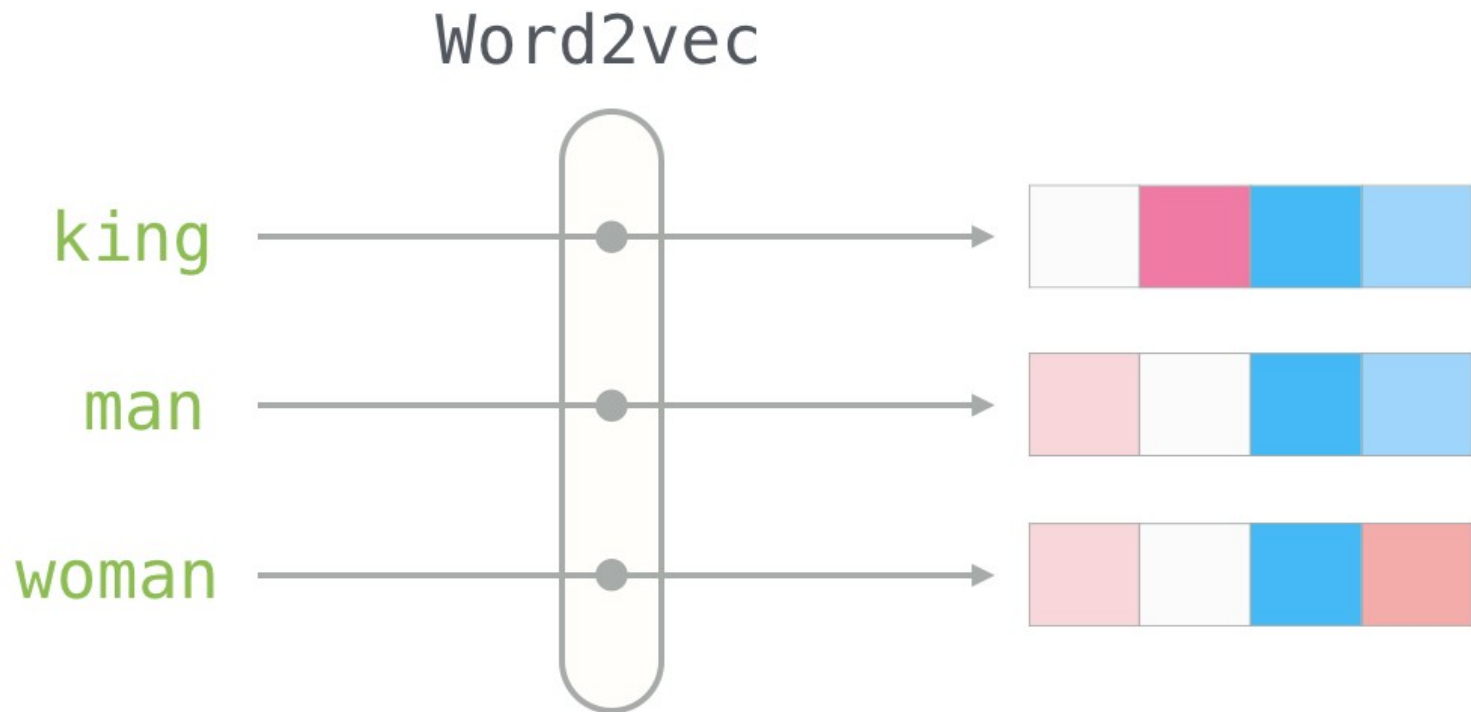
Introduction to Word Embeddings

- The Solution: Instead of a huge list of zeros, use a small list of decimal numbers (features).
- Example (Conceptual):
 - King: [0.9, -0.2, 0.5]
 - Queen: [0.8, -0.3, 0.6] (Notice: Similar numbers to King)
 - Apple: [0.1, 0.9, -0.1] (Very different numbers)
- Benefit: Words with similar meanings have similar mathematical vectors.
- Size: Dense vectors are small (e.g., 300 dimensions) compared to One-Hot (100,000 dimensions).

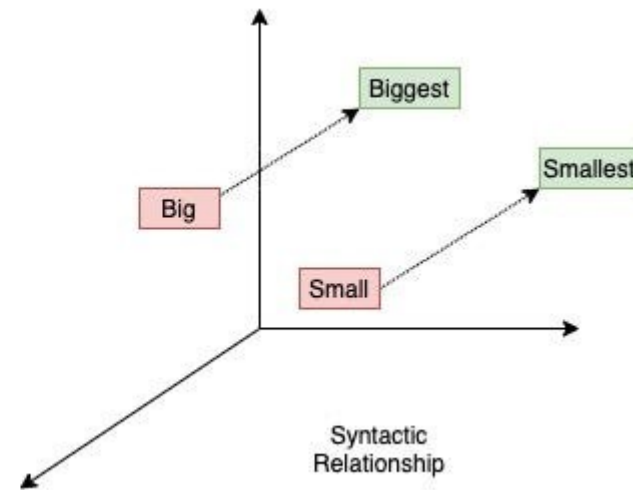
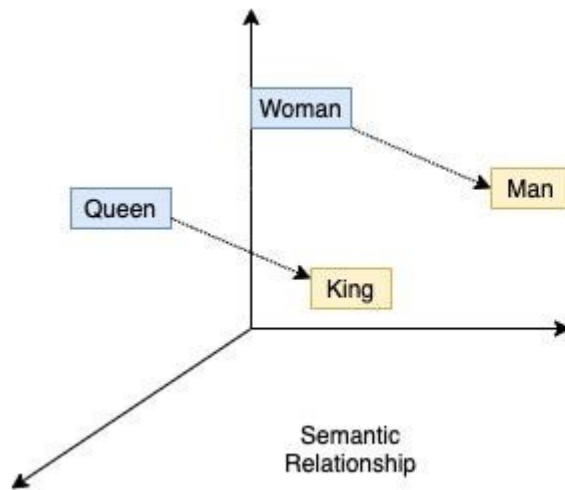
What is Word2Vec?

- Definition: A technique developed by Google (Tomas Mikolov et al.) in 2013 to generate word embeddings.
- It is NOT: Deep Learning (technically). It is a Shallow Neural Network (only 2 layers).
- Key Insight: It learns word meanings by looking at context.
- The Motto: "You shall know a word by the company it keeps." (J.R. Firth).

What is Word2Vec?



What is Word2Vec?



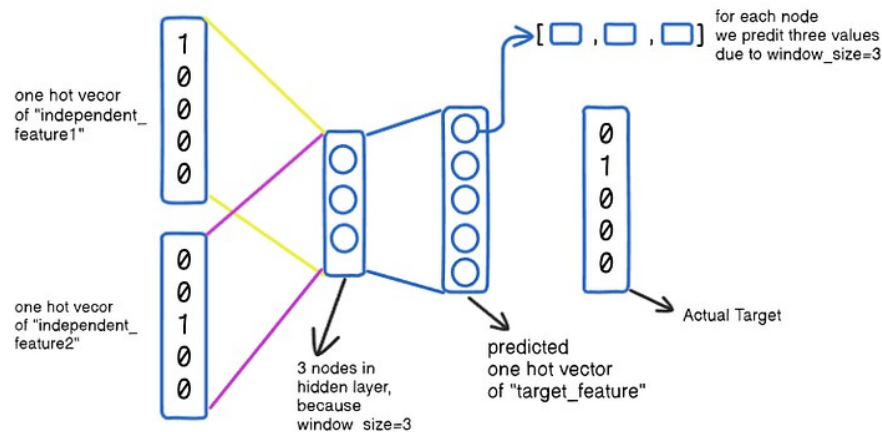
Two Ways to Learn

- Word2Vec isn't just one algorithm; it's a family of two distinct architectures. You choose one based on your data.
 - CBOW (Continuous Bag of Words)
 - Skip-gram

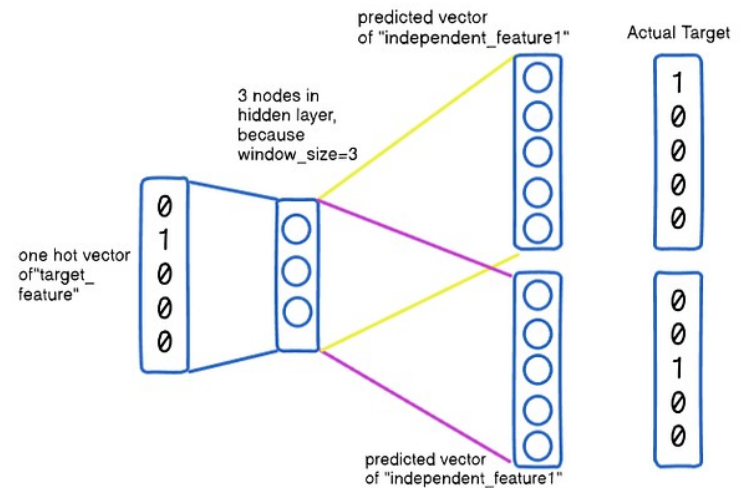
Two Ways to Learn

Word2vec

CBOW



Skip-gram



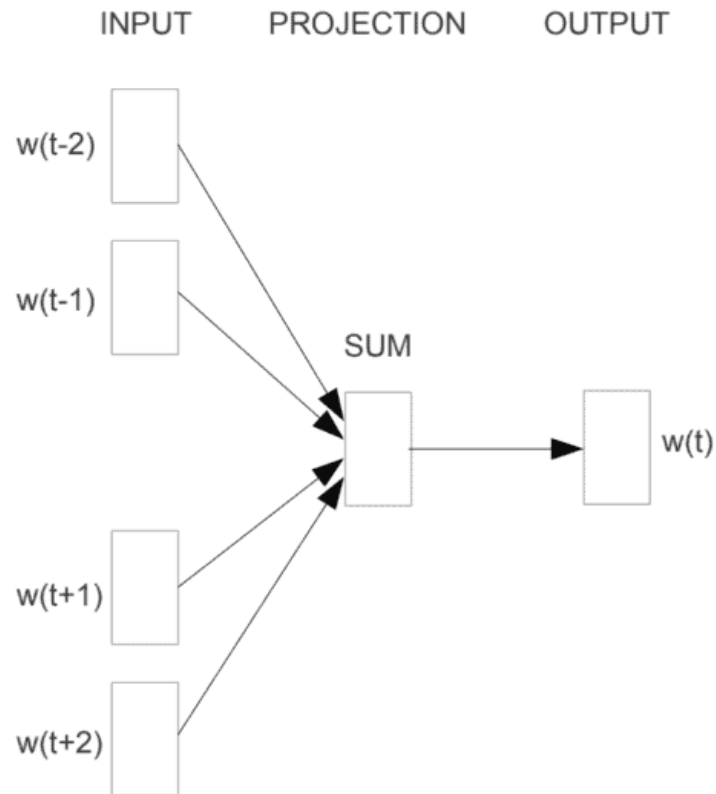
Continuous Bag of Words (CBOW)

- The Task: Fill in the blank.
- Mechanism: It takes the surrounding context words and tries to predict the middle (target) word.
- Example:
 - Context: "The", "quick", "fox", "jumps"
 - Target: "brown"
- Analogy: Like a student trying to guess a missing word in a sentence based on the clues around it.
- Best For: Faster training; better accuracy on frequent words.

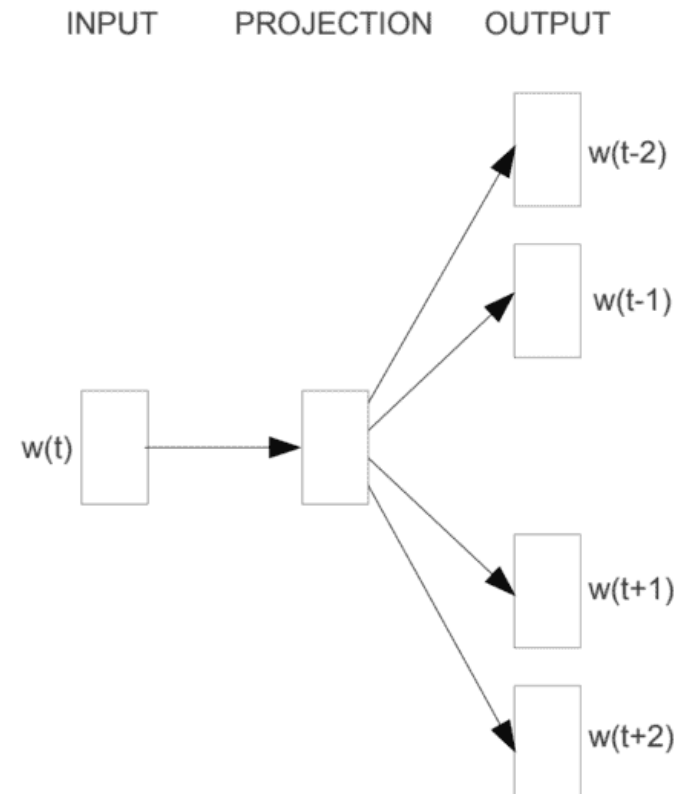
Skip-gram Model

- The Task: Create the context.
- Mechanism: It takes the middle (target) word and tries to predict the surrounding context words.
- Example:
 - Input: "brown"
 - Target Predictions: "The", "quick", "fox", "jumps"
- Analogy: Given a central topic, what other topics are likely to be discussed nearby?
- Best For: Works better with small datasets; represents rare words/phrases very well.

Differentiating



CBOW

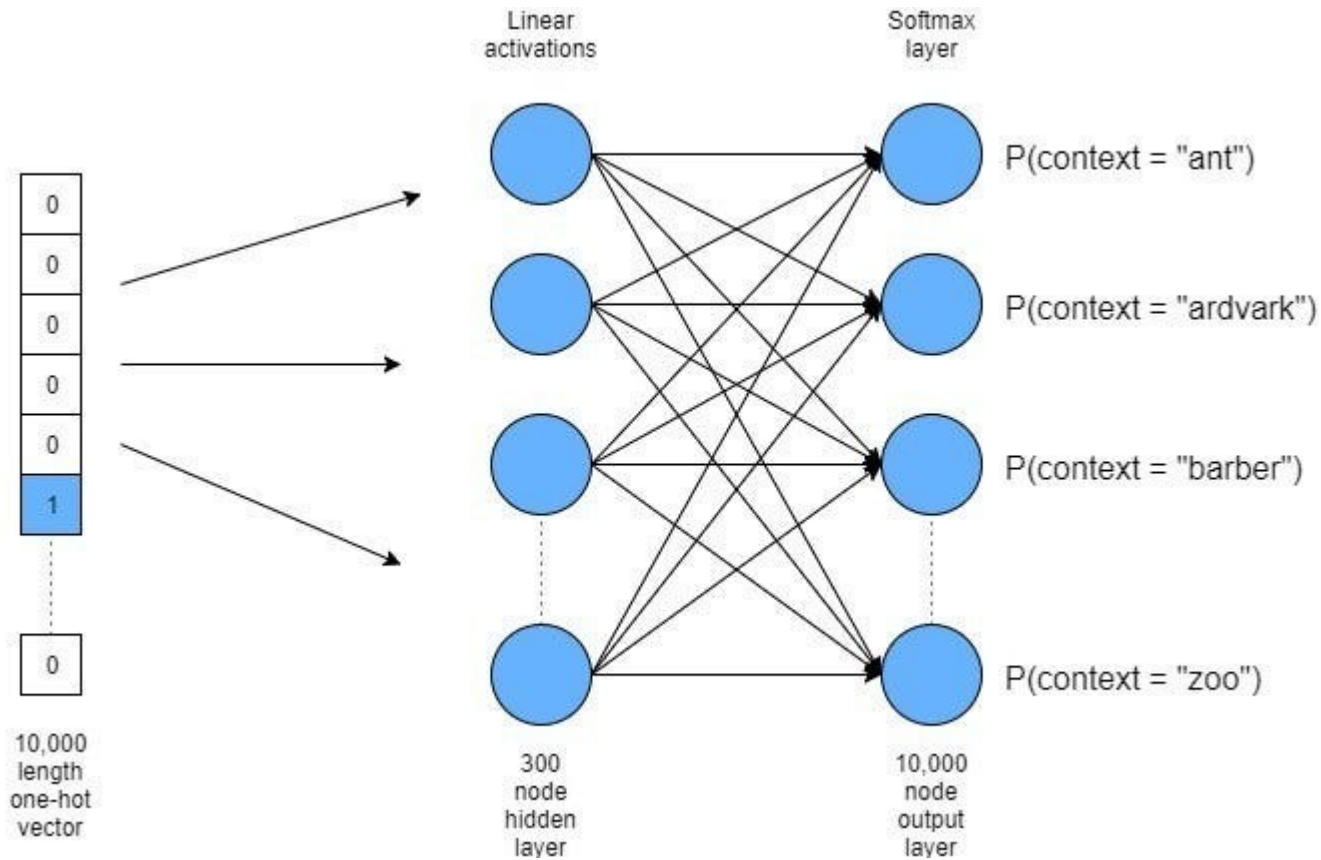


Skip-gram

The Neural Network Setup

- The Trick: We train a neural network to do a prediction task, but we don't care about the result.
- Setup:
 - Input Layer: One-hot encoded word.
 - Hidden Layer: Linear layer (No activation function like Sigmoid/ReLU). This is crucial!
 - Output Layer: Softmax probability of other words.
- The Secret: After training, we delete the Output Layer. The Hidden Layer weights become our final Word Vectors.

The Neural Network Setup



The Computational Bottleneck

- The Issue: The Output Layer has to calculate the probability for every word in the dictionary (e.g., 100,000 words) for every single training step.
- Result: This is incredibly slow.
- Solution: We need "shortcuts" to avoid calculating everything.

Making Word2Vec Fast

- Method 1: Hierarchical Softmax
 - Organizes words into a Binary Tree (Huffman Tree).
 - Common words are at the top, rare words at the bottom.
 - Reduces calculation from N to $\log_2(N)$.
- Method 2: Negative Sampling (Most Popular)
 - Instead of updating all 100,000 weights, we update the correct word (Positive sample) and just 5-10 random wrong words (Negative samples).
 - We ignore the rest of the dictionary for that step.

Semantic Algebra

- Once trained, the vectors exhibit algebraic relationships.
- The Classic Example:
 - $\text{Vector}(\text{King}) - \text{Vector}(\text{Man}) + \text{Vector}(\text{Woman}) \approx \text{Vector}(\text{Queen})$
- What this means: The model learned the concept of "Gender" and "Royalty" purely by reading text, without being explicitly taught.
- Other Examples:
 - $\text{Paris} - \text{France} + \text{Italy} \approx \text{Rome}$ (Capitals)
 - $\text{Walked} - \text{Walk} + \text{Swim} \approx \text{Swam}$ (Verb Tenses)

Summary

- Embeddings: Word2Vec converts words into dense vectors where similar words are close in space.
- Context is King: It learns by analyzing which words appear next to each other.
- Two Models: CBOW (Context $\rightarrow$ Word) and Skip-gram (Word $\rightarrow$ Context).
- Efficiency: Uses Negative Sampling to train on massive datasets quickly.

Word2Vec Example

- Practical

The Landscape Before GloVe (2014)

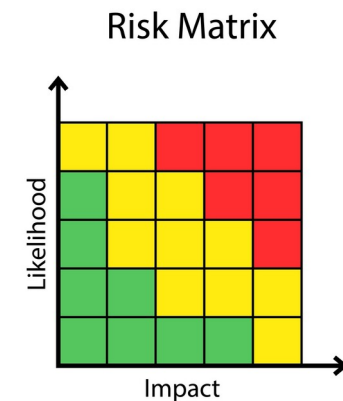
- Two main camps existed in NLP:
 - Count-Based Methods (e.g., LSA):
 - Used global statistics (Word-Document matrices).
 - Good at: Capturing similarity.
 - Bad at: Word analogies (King - Man + Woman = Queen).
 - Predictive Methods (e.g., Word2Vec):
 - Used local context windows.
 - Good at: Analogies and semantic tasks.
 - Bad at: Utilizing the full statistics of the corpus (it creates a new context window every time, forgetting the "big picture").

What is GloVe?

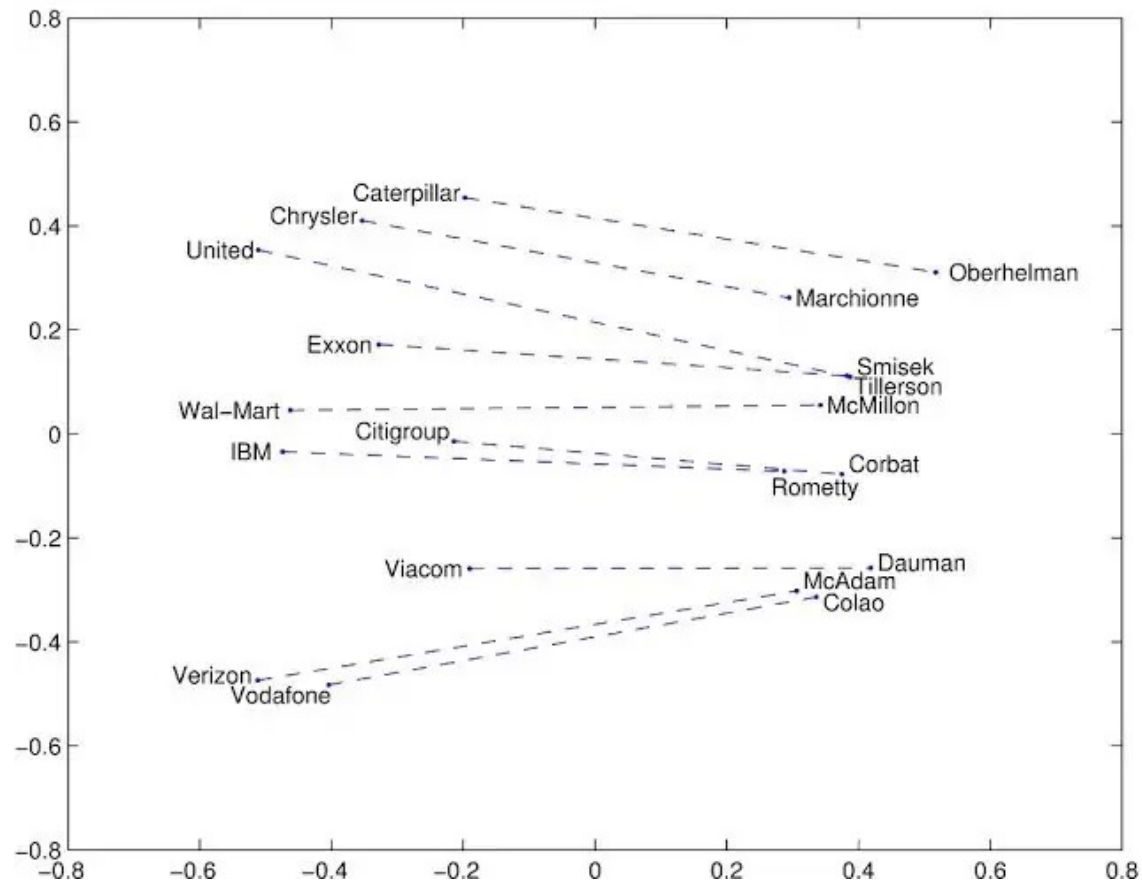
- Name: Global Vectors.
- Creators: Researchers at Stanford University (Pennington, Socher, Manning).
- The Big Idea: It aims to combine the benefits of both worlds:
 - The global statistical information of Matrix Factorization (LSA).
 - The linear algebraic properties of Word2Vec (Analogies).
- Core Concept: The ratio of co-occurrence probabilities is vital for learning meaning.

The Global Co-occurrence Matrix

- Before neural networks, GloVe starts with a simple table: the Matrix (X).
- X_{ij} : How many times word j appears in the context of word i .
- Example Corpus: "I like deep learning. I like NLP."
- Window Size: 1 (Left and Right).



GloVe



Comparison

Feature	Word2Vec (Skip-gram)	GloVe
Learning Method	Predictive (Neural Network)	Count-Based (Matrix Factorization)
Context	Local (Current window only)	Global (Entire corpus counts)
Training Speed	Fast	Generally Faster (Scales well)
Small Data	Better	Struggling
Analogies	Excellent	Excellent

Glove Example

- Practical

Seq2Seq Model

- Definition: A deep learning architecture designed to transform one sequence of data (Input) into another sequence of data (Output).
- The Key Characteristic: The length of the input sequence does not need to match the length of the output sequence.
- Example:
 - Input: "How are you?" (3 words)
 - Output: "Je vais bien" (3 words) OR "Ça va?" (2 words)

The Limitation of Standard RNNs

- Standard RNNs (Many-to-Many): Usually require 1-to-1 mapping.
 - Example: Named Entity Recognition (Input: 5 words → Output: 5 tags).
- The Real World Problem: Most NLP tasks are asynchronous.
 - Translation: Grammar differs between languages (SVO vs SOV).
 - Summarization: Input is long, output is short.
 - Chatbots: Question is short, answer might be long.
- Conclusion: We need a model that reads the whole input first, understands it, and then produces output.

Seq2Seq

Machine Language Translation

*Les modèles de séquence
sont super puissants*

Sequence Model

*Sequence models are super
powerful*

Text Summarization

*A strong analyst have 6
main characteristics. One
should master all 6 to be
successful in the industry :*

1.
2.

Sequence Model

*6 characteristics of
successful analyst*

Chatbot

How are you doing today?

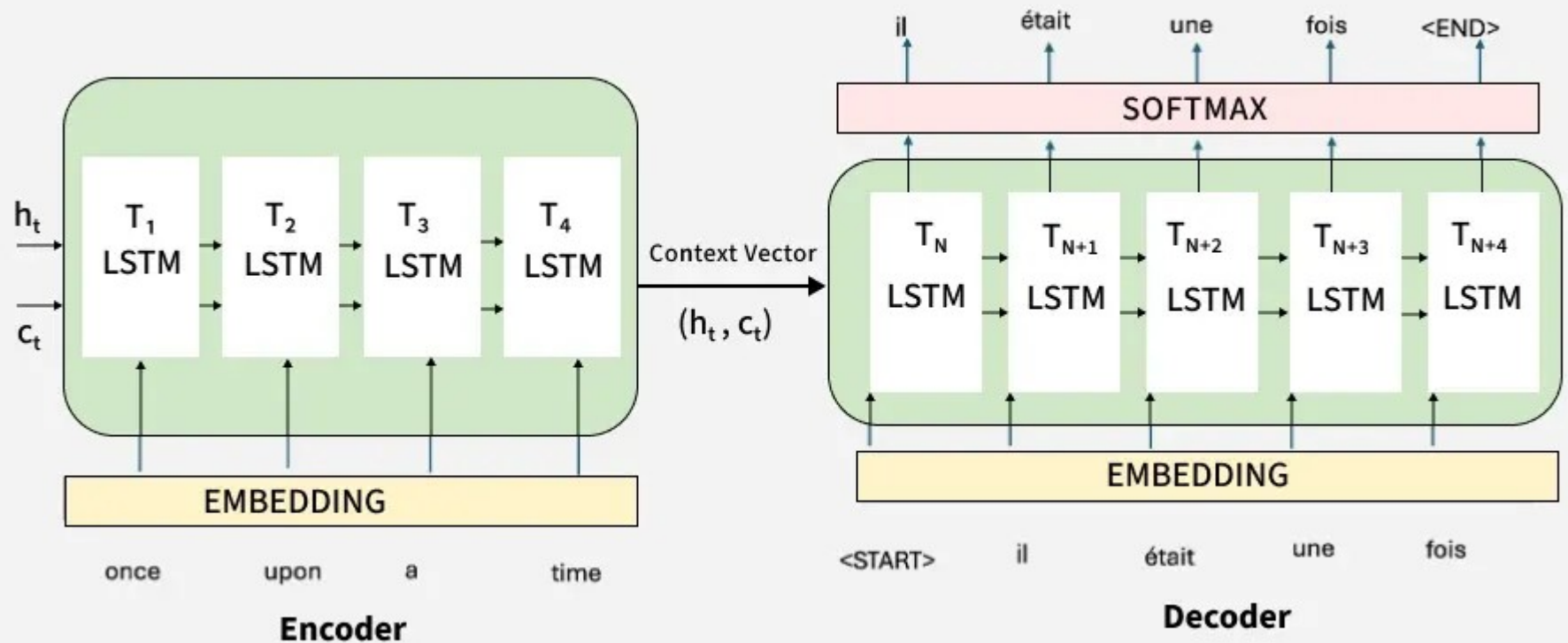
Sequence Model

*I am doing well. Thank you.
How are you doing today?*

The Encoder-Decoder Architecture

- Seq2Seq is often called the Encoder-Decoder network.
- It consists of two separate Recurrent Neural Networks (LSTMs or GRUs):
 - The Encoder: Reads the input sequence.
 - The Context Vector: A fixed-length vector that connects the two networks.
 - The Decoder: Generates the output sequence.

The Encoder-Decoder Architecture



The Encoder ("The Reader")

- Role: To understand the input sequence.
- Process:
 - It accepts the input sequence word-by-word (time step by time step).
 - At each step, it updates its Hidden State based on the new word and the previous state.
- Output: It discards the outputs at each step except for the Final Hidden State.
- Analogy: Listening to a sentence until the speaker finishes before starting to translate.

The Context Vector ("The Bottleneck")

- Definition: This is the Final Hidden State of the Encoder.
- Significance: It encapsulates the semantic meaning (the "thought") of the entire input sentence.
- Dimensions: It is a vector of floating-point numbers (e.g., size 256 or 512).
- The Handover: This vector is passed to the Decoder as its Initial State.

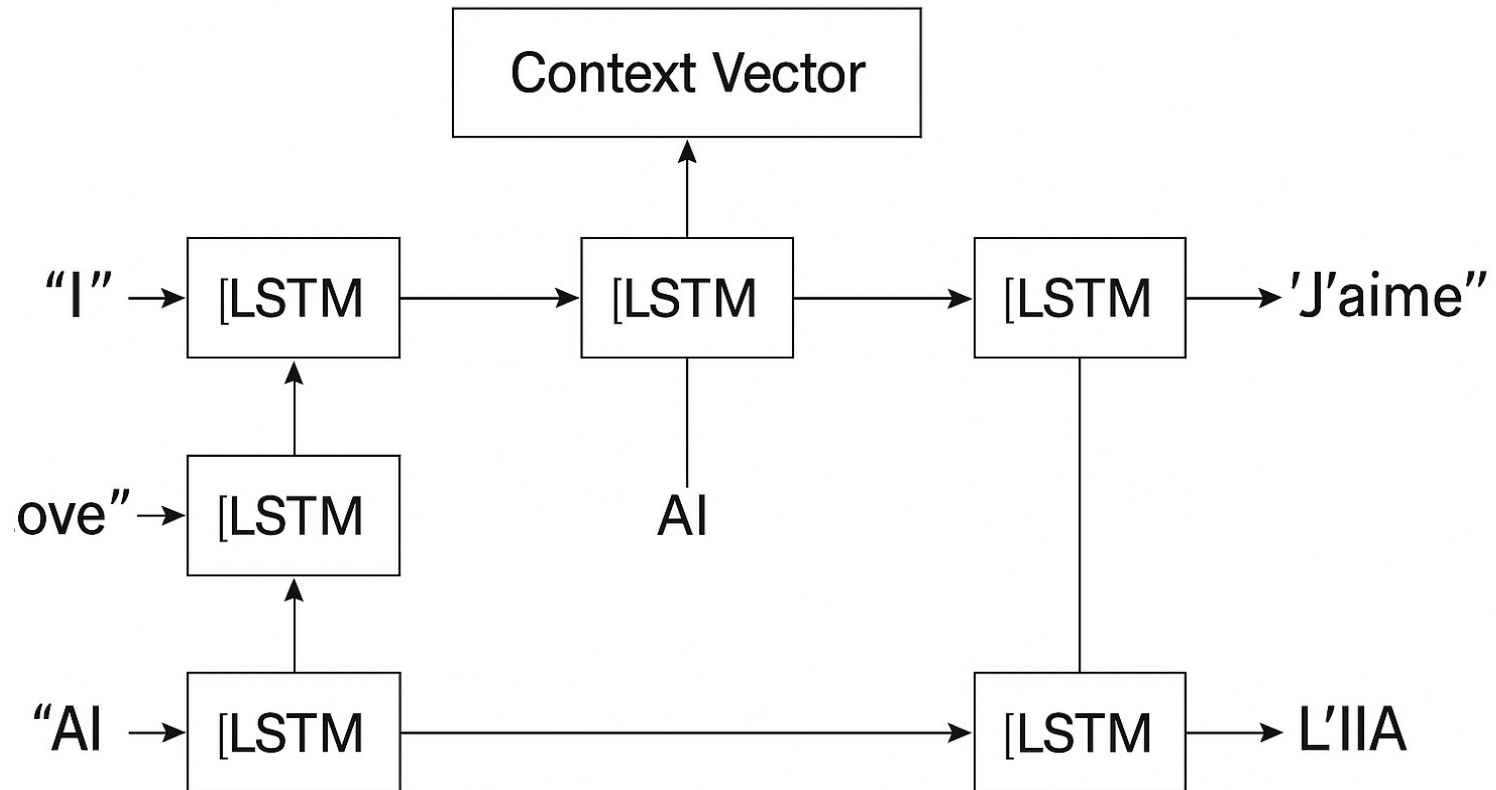
The Decoder ("The Writer")

- Role: To generate the output sequence.
- Initialization: It starts with the Context Vector (it inherits the "thought" from the Encoder).
- Process:
 - It receives a special <START> token to begin.
 - It predicts the first word.
 - It takes its own predicted word and feeds it back in as input for the next step.
- Termination: It stops generating when it predicts a special <END> token.

How It Learns (Teacher Forcing)

- Inference (Testing): The Decoder relies on its own previous prediction to guess the next word. (If it makes a mistake, the error propagates).
- Training: We use a technique called Teacher Forcing.
 - We don't feed the Decoder's predicted word into the next step.
 - Instead, we feed the actual correct word (Ground Truth) from the dataset.
- Benefit: It helps the model learn faster and converge more stably.

Flow



Where is Seq2Seq Used?

- Machine Translation: (Google Translate, DeepL).
- Text Summarization: Converting news articles into headlines.
- Chatbots / Conversational AI: Generating responses to user queries.
- Speech Recognition: Audio Spectrogram (Sequence) → Text Transcript (Sequence).
- Image Captioning: Image (processed by CNN) → Text Description.

The Information Bottleneck

- The Problem: The Context Vector has a fixed size.
 - If the input sentence is 100 words long, compressing all that meaning into one vector causes data loss.
 - The model tends to forget the beginning of long sentences.
- The Solution: Attention Mechanism (Bahdanau et al., 2014).
 - Allows the Decoder to "look back" at specific parts of the Encoder input at every step, rather than relying on just one static vector.
 - Note: This paved the way for Transformers.

Summary

- Seq2Seq maps sequences of different lengths.
- Encoder compresses input into a Context Vector.
- Decoder unfolds the Context Vector into the output.
- It revolutionized Machine Translation but struggles with very long sequences without Attention.

Seq-2-Seq Model Example

- Practical

Thank you

This presentation is created using LibreOffice Impress 7.4.1.2, can be used freely as per GNU General Public License



@mitu_skillologies



@mITuSkillologies



@mitu_group



@mitu-skillologies



@MITUSkillologies

kaggle

@mituskillologies

Web Resources

<https://mitu.co.in>

<http://tusharkute.com>



@mituskillologies

contact@mitu.co.in
tushar@tusharkute.com